

Taller 1 - Circuitos combinacionales con SystemVerilog y HDL Studio

Sistemas Digitales

Segundo Cuatrimestre 2026

1. Enunciado

Implementar circuitos combinacionales utilizando SystemVerilog y HDL Studio, componiendo módulos para obtener otros más complejos.

Para la realización de este taller deberá utilizar el paquete inicial del Taller 1 (devcontainer, esqueletos `.sv` y testbenches) disponible en la página de la materia.

Se deberá enviar al campus la resolución de los ejercicios marcados con **entrega obligatoria**: la carpeta `ejercicios/` completa (los Makefile apuntan a ejercicios anteriores). El plazo de entrega finaliza el 26 de agosto a las 23:59.

2. HDL Studio

2.1. El entorno

El trabajo se realiza dentro del entorno provisto por la materia (VS Code + devcontainer + HDL Studio). No es necesario instalar herramientas por fuera del contenedor.

El entorno opera en dos modos complementarios. Simulación y síntesis:

- En simulación se ejecuta un testbench, se observan entradas y salidas, y se comprueba el comportamiento del módulo.
- En síntesis se visualiza el circuito inferido a partir del código y se lo relaciona con las expresiones escritas.

2.2. Detalles adicionales

- a) Las entradas (`input`) se declaran en la interfaz del módulo y reciben valores desde el testbench o desde otros módulos.
- b) Las salidas (`output`) se declaran en la interfaz y deben recibir una asignación en el cuerpo del módulo.
- c) Las señales internas se declaran con `logic` dentro del módulo y conectan subexpresiones.
- d) No se deben cambiar los nombres de los archivos, ni sus puertos o señales, ya que de hacerlo dejarían de funcionar los testbenches y ejercicios sucesivos.
- e) Los testbenches son parte de la especificación: no deben modificarse para lograr que una implementación incorrecta sea aceptada.
- f) Cada módulo nuevo debe **instanciar** los módulos de ejercicios anteriores cuando el enunciado lo pida. No reimplementar esa lógica en un solo `assign`.
- g) Solo se puede usar la sintaxis vista en clase y la documentada en `sintaxis.typ`. No usar `for`, `generate`, `always`, `if/case` ni otra construcción que no esté en esa guía.

- h) `make sim` copia a la carpeta del ejercicio los `.sv` de ejercicios anteriores (fuente de verdad: el ejercicio de origen). No editar esas copias. Resolver en orden.
- i) Para sintetizar un módulo que instancia otros: después de `make sim`, agregar esos `.sv` (ya están en la misma carpeta) al circuito de HDL Studio. No agregar el testbench. *Create circuit* sobre un solo archivo no alcanza: Yosys tira `unknown module`. Detalle en `sintaxis.typ`.

3. Antes de empezar

1. Abrir la carpeta del taller en VS Code.
2. Aceptar la apertura dentro del devcontainer.
3. Verificar que HDL Studio y las extensiones de SystemVerilog estén activas.
4. Abrir el esqueleto de `ej1` y ejecutar su testbench (`make sim`).
5. Identificar en pantalla las entradas, las salidas y el circuito sintetizado.

Si el entorno no funciona, registrar el mensaje completo del error y consultar antes de instalar herramientas por fuera del contenedor.

4. Ejercicios

Completar el esqueleto provisto de los módulos que se enumeran a continuación. Deberán diseñar cada uno de manera modular, incorporando una funcionalidad por módulo.

Cuando un ejercicio reutiliza un módulo anterior, `make sim` copia ese `.sv` a la carpeta. Completar primero el ejercicio de origen y no editar la copia.

4.1. Ejercicio 1: Sumador simple

En la carpeta `ejercicios/ej1`, implementar un sumador de 1 bit sin acarreo de entrada (*half-adder*). Suma `a` y `b`. Produce la suma `sum` y el acarreo `cout`.

Tabla de verdad:

a	b	sum	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
module sumador_simple (  
    input  logic a,  
    input  logic b,  
    output logic sum,  
    output logic cout  
);  
    // completar los assign  
endmodule
```

- Escribir las asignaciones. Se puede usar `^` y `&`, o solo AND/OR/NOT.
- Ejecutar el testbench (`make sim`) y comprobar que los 4 casos pasan.
- Sintetizar en HDL Studio e identificar las dos salidas.

4.2. Ejercicio 2: Sumador completo

En la carpeta `ejercicios/ej2`, implementar un sumador de 1 bit con acarreo de entrada (*full-adder*) usando dos `sumador_simple`. `make sim` copia `sumador_simple.sv` del ejercicio 1 a esta carpeta.

El sumador completo suma los bits `a`, `b` y `cin`. Produce `sum` y `cout`.

```
module sumador_completo (  
    input  logic a,  
    input  logic b,  
    input  logic cin,
```

```

        output logic sum,
        output logic cout
    );
    // reutilizar el sumador simple para esta implementación
endmodule

```

- Completar el módulo.
- Ejecutar `make sim` (8 combinaciones de `a`, `b`, `cin`).
- Verificar que `sumador_simple.sv` se haya copiado en la carpeta del ej2 tras `make sim`.
- Sintetizar y verificar la solución.

Checkpoint: half-adder + full-adder

4.3. Ejercicio 3: Sumador de 4 bits — entrega obligatoria

En la carpeta `ejercicios/ej3`, implementar un sumador con carry de 4 bits utilizando el módulo `sumador_completo`. `make sim` copia `sumador_completo.sv` y `sumador_simple.sv` de los ejercicios anteriores a esta carpeta.

El sumador de 4 bits sumará los números enteros sin signo de 4 bits `a` y `b`, un carry de entrada `cin`, y devolverá 4 bits con el resultado en `sum` y un carry de salida `cout`.

```

module sumador_4b (
    input  logic [3:0] a,
    input  logic [3:0] b,
    input  logic      cin,
    output logic [3:0] sum,
    output logic      cout
);
    // resolver utilizando sumador_completo
endmodule

```

- Completar el módulo.
- Calcular a mano `a = 4'b0111`, `b = 4'b0001`, `cin = 0`. ¿Cuál es `sum` y `cout`?
- Ejecutar `make sim`. Verificar que se hayan copiado todos los archivos bien.
- Sintetizar y recorrer la cadena de acarreo.

4.4. Ejercicio 4: Restador simple

En la carpeta `ejercicios/ej4`, implementar un restador de 1 bit sin préstamo de entrada (*half-subtractor*). Calcula $a - b$. Produce la diferencia `diff` y el préstamo `bout` (*borrow*).

Completar la tabla.

a	b	diff	bout
0	0		
0	1		
1	0		
1	1		

A partir de la tabla de verdad, escribir las ecuaciones esperadas:

$$\text{diff} = \dots \quad \text{bout} = \dots$$

Implementar las ecuaciones encontradas en SystemVerilog:

```
module restador_simple (  
    input logic a,  
    input logic b,  
    output logic diff,  
    output logic bout  
);  
    // completar los assign  
endmodule
```

- Completar la tabla a mano, escribir las ecuaciones y después completar el código del módulo.
- Ejecutar `make sim`.
- Sintetizar y comparar con el sumador simple.

4.5. Ejercicio 5: Restador completo

En la carpeta `ejercicios/ej5`, implementar un restador de 1 bit con préstamo de entrada **usando** `restador_simple`. `make sim` copia `restador_simple.sv` del ejercicio 4 a esta carpeta.

Calcula $a - b - \text{bin}$. Produce `diff` y `bout`.

```
module restador_completo (  
    input logic a,  
    input logic b,  
    input logic bin,  
    output logic diff,  
    output logic bout  
);  
    // completar el módulo  
endmodule
```

- Completar el módulo.
- Ejecutar `make sim`.
- Sintetizar y verificar la jerarquía.

4.6. Ejercicio 6: Restador de 4 bits — entrega obligatoria

En la carpeta `ejercicios/ej6`, implementar un restador con borrow de 4 bits usando el `restador_completo`. `make sim` copia `restador_completo.sv` y `restador_simple.sv` de los ejercicios anteriores a esta carpeta.

```
module restador_4b (  
    input logic [3:0] a,  
    input logic [3:0] b,
```

```

    input logic      bin,
    output logic [3:0] diff,
    output logic      bout
);
    // completar
endmodule

```

El préstamo entra por el bit 0 (*bin*) y se encadena hasta *bout*.

- Completar el módulo.
- Calcular a mano $a = 4'b0100$, $b = 4'b0001$, $bin = 0$.
- ¿Qué pasa si *b* es mayor que *a*?
- Ejecutar `make sim`.
- Sintetizar y recorrer la cadena de *borrow*s.

Checkpoint: restador simple, completo y de 4 bits

4.7. Ejercicio 7: Inversor condicional

En la carpeta ejercicios/ej7, implementar un circuito que toma un bus de 4 bits x y un bit de control $ctrl$. Si $ctrl = 0$, la salida y copia x . Si $ctrl = 1$, y es x invertido bit a bit.

```
module inversor_cond (
    input logic [3:0] x,
    input logic      ctrl,
    output logic [3:0] y
);
    // completar
endmodule
```

TIP: ¿Conocemos una compuerta de dos entradas a y b , que invierta a según el valor de b ? Si la conocemos, podemos utilizar la sintaxis de réplica:

```
assign y = x <MI_COMPUERTA> {4{ctrl}};
```

$\{4\{ctrl\}\}$ replica $ctrl$ cuatro veces: $\{ctrl, ctrl, ctrl, ctrl\}$. Otra opción: el condicional `assign y = ctrl ? ~x : x`.

- Completar el módulo.
- Ejecutar `make sim`.
- Sintetizar y revisar el circuito.

4.8. Ejercicio 8: Sumador/restador de 4 bits — entrega obligatoria

En la carpeta ejercicios/ej8, implementar un circuito que, dados dos números de 4 bits a y b y un bit de control $ctrl$, calcule:

- $ctrl = 0$:

$$y = a + b$$

, y `cout` es el acarreo

- $ctrl = 1$:

$$y = a - b$$

, y `cout` es el préstamo

Hay que **reutilizar** `sumador_4b` y `restador_4b` (con `cin` y `bin` en 0). Elegir entre suma y resta con el condicional `?` (multiplexor). `make sim` copia esos `.sv` de los ejercicios 3 y 6 a esta carpeta.

```
module sumador_restador_4b (
    input logic [3:0] a,
    input logic [3:0] b,
    input logic      ctrl,
    output logic [3:0] y,
    output logic      cout
);
    // completar
endmodule
```

- Completar utilizando los sumadores y restadores de 4 bits y un `?`.
- Ejecutar `make sim`.
- Sintetizar y señalar en el circuito el sumador, el restador y el multiplexor.

Checkpoint: add/sub de 4 bits por composición

4.9. Ejercicio 9: Solo AND, OR y NOT

En la carpeta ejercicios/ej9, se da la fórmula

$$f = (x + y) \cdot (x + \overline{y}) \cdot (x + z)$$

que equivale a

x

. Implementarla usando solo AND, OR y NOT (sin XOR).

```
module formula_and_or_not (  
    input logic x,  
    input logic y,  
    input logic z,  
    output logic f  
);  
    // completar con &, | y ~  
endmodule
```

- Armar la tabla de verdad de 8 filas de la fórmula original y comprobar la equivalencia.
- Implementar la fórmula tal cual y sintetizar.
- Reimplementar la fórmula reducida y sintetizar.
- Comparar ambos circuitos.

Importante: Los ejercicios 3, 6 y 8 son de entrega obligatoria. Entregar la carpeta `ejercicios/` completa: los `Makefile` apuntan a los ejercicios anteriores.